# Automated Irregular Shaped Object Stacking via Pose Optimization and Monte Carlo Tree Search

Author Names Omitted for Anonymous Review. Paper-ID [add your ID here]

***Abstract*—Building freestanding structures from irregular objects requires reasoning about both contact stability and the effects of each placement on subsequent construction. We study automated stacking from a finite inventory of objects to approximate a prescribed target shape without support from container walls. We propose a planning framework that combines bilevel pose optimization with Monte Carlo Tree Search (MCTS). The pose optimizer refines geometric placement proposals by coupling target-shape and collision objectives with contact-force optimization that evaluates force and torque equilibrium across the provisional structure under friction constraints. A bodywise decomposition of the force subproblem enables parallel computation on the GPU. Physical simulation evaluates candidate placements and produces settled successor states for a batched, receding-horizon MCTS planner, which searches over object choices and placement poses. This formulation connects continuous placement refinement with the consequences of multiple successive placements. We hypothesize that combining structural equilibrium evaluation with multi-step search improves target coverage while maintaining stability under a fixed planning budget.**

## I. Introduction

Automated packing and palletizing have long been studied to improve space utilization and material handling in logistics and manufacturing [6]. Recent robotic packing methods accommodate irregular objects while considering packing density, stability, and placement feasibility [22, 9]. Constructing freestanding structures from natural stones requires finding mutually supporting arrangements among uneven surfaces while achieving a prescribed target shape without support from container walls.

Several stone-stacking methods use placement criteria developed for particular construction settings, including tower-building objectives [7] and masonry-inspired geometric rules [12, 19]. For example, Johns et al.'s double-faced wall planner generates candidates along the intersection of the target wall surfaces and the existing structure, and applies inward-slope heuristics tailored to wall construction [12]. The ability to construct freeform walls [13] demonstrates flexibility in wall geometry, while extending beyond such construction settings motivates planning driven by target geometry without wall-specific placement rules. Structural stability also requires reasoning beyond the support of the newly placed object: its load can affect the equilibrium of objects beneath it. Liu and Napp [17] explicitly address structural strength during planning, demonstrating the value of evaluating the structure as a whole. However, assessing the current structure alone does not determine how a placement affects future support and space utilization. Menezes et al. [20] address sequential placement through learned action values that account for future rewards, with a prescribed object sequence and fixed orientations. We investigate explicit lookahead over object choices and placement poses, coupled with structural equilibrium evaluation and target-shape objectives.

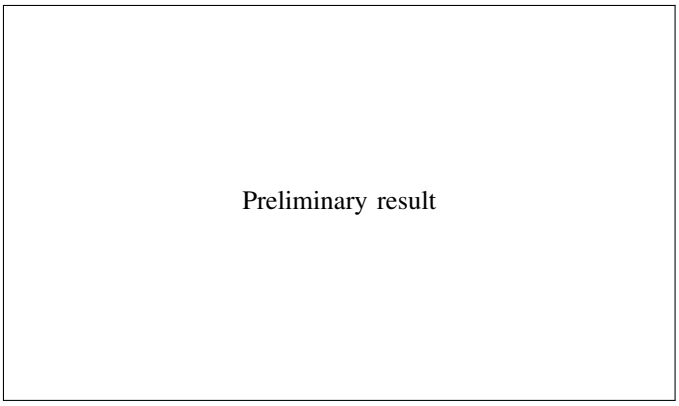



Fig. 1: Preliminary result.

We consider stacking a finite inventory of $n$ irregular objects to approximate a target region $T \subset \mathbb{R}^3$. Let $O_i(q_i) \subset \mathbb{R}^3$ denote the volume occupied by object $i$ at pose $q_i \in \mathrm{SE}(3)$. The final configuration objective is

$$\min_{q_1,\cdots,q_n \in \mathrm{SE}(3)} \mathbb{V}\left(T \setminus \bigcup_{i=1}^{n} O_i(q_i)\right) \quad \text{s.t.} \quad c_{\text{phys}}(O_1(q_1),\cdots,O_n(q_n)) = 0, \tag{1}$$

where $\mathbb{V}(\cdot)$ denotes volume and $c_{\text{phys}} = 0$ represents stability and non-interpenetration constraints. Jointly optimizing all object poses under coupled physical constraints is computationally demanding. To obtain a practical planning procedure, we decompose construction into sequential decisions, selecting one object and optimizing its placement at each step while checking the resulting structure's physical feasibility. This decomposition reduces the dimension of each pose optimization, but each placement changes the support geometry, free space, and remaining inventory. Multi-step lookahead allows the planner to evaluate these future consequences beyond immediate placement quality. Our central question is how to combine this lookahead with structural equilibrium evaluation in a common framework for different target shapes.

We address this question with a framework coupling bilevel pose optimization and Monte Carlo Tree Search (MCTS). The pose optimizer refines geometric proposals using target-shape

objectives and a physical objective that evaluates force and torque equilibrium across the structure under friction constraints. The target enters through its geometric representation, allowing the same optimization formulation to be applied to different target shapes. A bodywise ADMM decomposition enables parallel solution of the contact-force subproblem. The sequence planner embeds pose refinement and physical simulation in batched, receding-horizon MCTS, following prior work on batched simulation-based search [8]. Simulation supplies settled successor states and rejects unstable placements, while multi-step lookahead compares their effects on subsequent construction.

We hypothesize that combining structural equilibrium evaluation with multi-step lookahead improves target coverage while maintaining stability across different target geometries under a fixed planning budget. Comparisons with greedy placement and candidate-only equilibrium evaluation will test the effects of lookahead and structural coupling, respectively. Evaluations across target shapes and search horizons will examine the applicability of the common formulation and its computational tradeoffs. The proposed contributions are:

- A bilevel placement optimizer that combines explicit target geometry with contact-force optimization across the structure, using bodywise ADMM updates for GPU computation.
- A batched, receding-horizon stacking planner that integrates pose refinement and physical simulation with multi-step lookahead over object choices and placement poses.

## II. Related Works

### A. Automated Stone Stacking

Furrer et al. [7] combined gradient-based pose search with physics simulation for stone stacking. Johns et al. [12] integrated perception, placement planning, and excavator manipulation for wall construction, later combining constrained registration and signed-distance-field classification to plan prescribed wall geometries [13]. Liu et al. [19] selected simulation-generated poses using hierarchical filters informed by masonry heuristics. Liu and Napp [17] incorporated structural strength evaluation into sequence planning and validated simulated predictions through physical wall tests. Their subsequent work introduced stackability measures for stone selection in corbelled shelter construction [18]. Menezes et al. [20] learned placement value maps from elevation maps using deep Q-learning, with rewards accounting for target occupancy and object displacement. Their policy selects planar positions for a prescribed object sequence with fixed orientations; its observed construction strategies inform our initial pose sampler.

### B. Irregular Object Packing

Wang and Hauser [22] used heightmap minimization to pack nonconvex objects while considering stability and robot motion feasibility, with conservative planning and replanning to address execution errors. Huang et al. [9] jointly planned object order and placement through hierarchical reinforcement learning using multi-view heightmaps and rewards for compactness, pyramidality, and simulated stability. Such geometric guidance is relevant to generating initial placement candidates before continuous pose refinement.

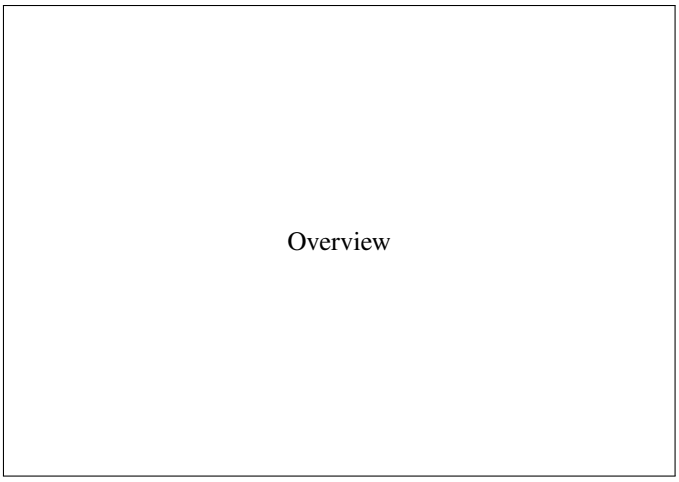



Fig. 2: Overview.

### C. Simulation-Based Sequential Manipulation Planning

King et al. [14] extended MCTS to nonprehensile rearrangement under uncertainty using learned and heuristic rollout policies. Huang et al. [8] introduced Parallel Monte Carlo Tree Search with Batched Simulations (PMBS), coupling parallel search with GPU rigid-body simulation for object retrieval from clutter. Our stacking planner adopts batched evaluation, combining geometric sampling, equilibrium-aware pose refinement, and structural simulation at each expansion.

## III. Overview and Preliminaries

### A. Shape Representation

Throughout the paper, we use the union of differentiable support function (DSF) to represent the 3D shapes of the target shape $T$ and the objects $O_i$, $i \in \mathcal{I}_k$, in the inventory. Given a compact convex body $C \subset \mathbb{R}^3$, the support function $h_C : \mathbb{R}^3 \backslash \{0\} \to \mathbb{R}$ is defined as

$$h_C(x) = \max_{s \in C} x \cdot s$$

where $\cdot$ denotes the standard inner product in $\mathbb{R}^3$. The work of Lee et al. [15] presented a differentiable version of the support function $h_C$ that represents a smooth shape $C$.

The function $h_C(x)$ can be easily extended through rigid-body transformation as follows:

$$h_{\bar{C}(q)}(x) = h_C(R^\top x) + t \cdot x$$

for $q := (R, t) \in \mathrm{SE}(3)$ where $\bar{C}(q) = \{Rp + t \mid p \in C\}$ denotes a transformed $C$ under rigid-body transformation. Given two DSF represented convex bodies $C_1(q_1)$ and $C_2(q_2)$ with SE(3) poses $q_1, q_2$, the signed distance $\mathrm{d}(C_1(q_1), C_2(q_2))$ between the two convex bodies can be easily computed using

Riemannian optimization techniques associated with $h_{C_1(q_1)}$ and $h_{C_2(q_2)}$ as presented in An, et al. [2]. Moreover, due to the smoothness of DSFs, the signed distance $\mathrm{d}(C_1(q_1), C_2(q_2))$ and the other contact information (e.g., normal vectors and contact points) are differentiable with respect to $q_1$ and $q_2$, which allows us to use gradient-based optimization techniques.

### B. Markov Decision Process Formulation

To address the construction objective in Eq. (1), we model stacking as a finite-horizon sequential decision process in which one object is placed at each step. Let $\mathcal{X}_k = \{(i, q_i) \mid i \in \mathcal{P}_k\}$ be the current scene and $\mathcal{I}_k = \{1, \ldots, n\} \setminus \mathcal{P}_k$ the remaining inventory. The state, action, and deterministic transition function are

$$s_k = (\mathcal{X}_k, \mathcal{I}_k, T), \quad a_k = (i_{k+1}, q_{k+1}), \quad s_{k+1} = F(s_k, a_k),$$

where $i_{k+1} \in \mathcal{I}_k$. The transition inserts the proposed pose into the scene, simulates the resulting structure, and returns the settled scene with $\mathcal{I}_{k+1} = \mathcal{I}_k \setminus \{i_{k+1}\}$. The reward $r(s_k, a_k)$ measures construction progress and physical validity; its implementation is described in Sec. V. The sequence planner therefore operates on the MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, F, r, \gamma)$.

### C. Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) incrementally constructs a search tree $\mathcal{T} = (\mathcal{N}, \mathcal{E})$ over the MDP. Each node $n \in \mathcal{N}$ represents a state $s(n) \in \mathcal{S}$, and each edge $(n, a) \in \mathcal{E}$ represents an action $a \in \mathcal{A}$ and its transition to a child node. For a node or edge, let $N(n)$ denote the number of completed evaluations and let $Q(n)$ denote the empirical mean of the discounted returns backed up to that node. An action prior $P(n, a)$, distinct from the transition function $F$, can provide additional guidance when the action space is large or continuous.

Each MCTS iteration consists of four conceptual operations. During *selection*, the tree policy chooses a promising child by balancing its empirical value and an exploration bonus. During *expansion*, a new action is generated and attached to the selected node. During *evaluation*, the transition associated with the action is evaluated to obtain a reward and a successor state. Finally, during *backup*, the discounted return is propagated through the selected path to update $Q(n)$ and $N(n)$. A nonterminal leaf may additionally be assigned a leaf-value estimate $\hat{V}(s)$, typically obtained from rollout simulation or a problem-specific heuristic.

In the present problem, an action is a placement of one remaining object, and its evaluation includes pose refinement and physical simulation. The geometry-informed proposals, batched selection, and finite-horizon leaf evaluation used to adapt MCTS to this expensive continuous-action setting are described in Sec. V.

### D. Overall Structure of the Sequence Planner

The overall structure of the sequence planner is illustrated in Fig. 2. The sequence planner uses Monte Carlo Tree Search (MCTS) to explore the space of possible stacking sequences. For each expansion, it first uses the initial pose sampler $\pi_{\text{init}}$ to generate a candidate pose $\hat{q}_{k+1}$, and then uses the pose optimization module $\pi_{\text{opt}}$ to refine it into a locally optimal pose $q_{k+1}$. The structural simulation module $\pi_{\text{sim}}$ subsequently implements the physical transition by evaluating the entire provisional scene and producing the settled next state. The resulting validity and simulation statistics are used to determine whether the action is admitted to the search tree and to evaluate its reward.

This gives the following execution flow:

$$s_k \xrightarrow{\pi_{\text{init}}} \hat{q}_{k+1} \xrightarrow{\pi_{\text{opt}}} q_{k+1} \xrightarrow{\pi_{\text{sim}}} s_{k+1}, \; r(s_k, a_k).$$

We describe pose optimization in Sec. IV and sequence planning, including structural simulation and reward design, in Sec. V.

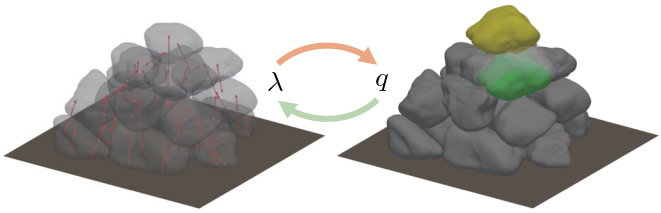



Fig. 3: Bilevel pose optimization. (Left) For a fixed candidate pose $q$, the lower-level problem optimizes contact forces $\lambda$ under friction constraints to minimize structural equilibrium residuals. (Right) The upper-level problem refines $q \in \mathrm{SE}(3)$ using the geometric objective and the optimized physical objective.

## IV. Pose Optimization

Before presenting the other modules of our proposed sequence planner, we first introduce the pose optimization module $\pi_{\text{opt}}$ that refines the candidate pose $\hat{q}_{k+1}$ generated by the initial pose sampler $\pi_{\text{init}}$ to find a locally optimal placement pose $q_{k+1}$ that is physically feasible and stable.

### A. Single Object Pose Optimization Problem

Given $s_k$ and the candidate pose $\hat{q}_{k+1}$ generated by $\pi_{\text{init}}$, the pose optimization module $\pi_{\text{opt}}$ solves the following problem to obtain a locally optimal pose $q_{k+1}$:

$$\begin{aligned} \min_{q} \quad & \left[ f_{\text{geo}}(q) + \min_{\lambda} f_{\text{phys}}(q, \lambda) \right] \\ \text{s.t.} \quad & g_{\text{cone},j}(q, \lambda_j) \le 0, \quad j = 1, \cdots, n_c \end{aligned} \tag{2}$$

where $q \in \mathrm{SE}(3)$ is the pose of the object and $\lambda = (\lambda_1, \cdots, \lambda_{n_c}) \in \mathbb{R}^{3n_c}$ is the vector of contact forces at the $n_c$ contact points between the object and the scene. This formulation follows prior contact-aware optimization methods [16, 5].

The geometric objective $f_{\text{geo}}(q)$ measures geometric quality, including target shape approximation and collision avoidance; its detailed formulation is provided in the appendix. The physical objective $f_{\text{phys}}(q, \lambda)$ is designed to evaluate whether

the contact forces $\lambda$ are physically feasible and contribute to a stable configuration. For indices $l, m \in \mathcal{P}_{k+1} := \mathcal{P}_k \cup \{i_{k+1}\}$ such that $l \neq m$, let $I_{l,m} \subset \{1, \cdots, n_c\}$ be the set of contact point indices associated with the objects $O_l$, $O_m$, and let $I_l := \bigcup_{m \in \mathcal{P}_{k+1} \setminus \{l\}} I_{l,m}$ be the set of contact point indices associated with the object $O_l$. Then, the physical objective $f_{\text{phys}}(q, \lambda)$ is defined as the following:

$$\begin{aligned} f_{\text{phys}}(q, \lambda) = w_{\text{eq}} \sum_{l \in \mathcal{P}_{k+1}} \| J_{c,I_l}^{\top}(q) \lambda_{I_l} + \lambda_{\text{ext},l} \|^2 \\ + w_{\text{cp}} \sum_{j \in I_{i_{k+1}}} \mathrm{d}_j(q)^2 \|\lambda_j\|^2 + \rho \sum_{j=1}^{n_c} \|\lambda_j\|^2 \end{aligned} \tag{3}$$

The three terms respectively penalize violations of static equilibrium, contact complementarity between the candidate and the current scene, and large contact forces through regularization. Here, $\mathrm{d}_j(q)$ is the signed distance between the two bodies associated with contact candidate $j$. $J_{c,I_l}$ is the contact Jacobian for object $O_l$, $\lambda_{\text{ext},l}$ is its external wrench, $\lambda_{I_l}$ is the vector of contact forces at the contact points in $I_l$, and $w_{\text{eq}}, w_{\text{cp}}, \rho > 0$ are weighting and regularization parameters.

Finally, each $g_{\text{cone},j}(\cdot)$ is the smoothed friction cone constraints for the $j$-th contact point defined by

$$g_{\text{cone},j}(q, \lambda_j) = \sqrt{\|J_{c,j}^t \lambda_j\|^2 + \epsilon} - \mu_j J_{c,j}^n \lambda_j \tag{4}$$

where $\epsilon > 0$ is a smoothing parameter, $\mu_j$ is the friction coefficient, and $J_{c,j}^t$ and $J_{c,j}^n$ are the tangential and normal components of the Jacobian of the contact constraint.

We note that, due to the smoothness of DSFs, the terms in the objective function dependent to $q$ are formulated which allows us to use gradient-based optimization techniques.

### *B. Upper-Level Problem Optimization on SE(3) Manifold*

Denoting $\lambda^*(q)$ as the optimal $\lambda$ of the lower-level problem for a given pose $q$, we define the force dependent objective function as $f_{\text{phys}}(q) := f_{\text{phys}}(q, \lambda^*(q))$. Then, using the *Envelope theorem* [4], we can compute the gradient of $f_{\text{phys}}(q)$ with respect to $q$ as follows:

$$\frac{df_{\text{phys}}(q)}{dq} = \frac{\partial f_{\text{phys}}(q, \lambda^*)}{\partial q} + \sum_{j=1}^{n_c} \nu_j^* \frac{\partial g_{\text{cone},j}(q, \lambda_j^*)}{\partial q}$$

where $\nu_j^*$ is the optimal dual variable associated with the $j$-th friction cone constraint $g_{\text{cone},j}(q, \lambda_j^*) \leq 0$. Using the above gradient, we optimize the placement pose $q$ on SE(3) manifold based on the Riemannian trust-region method [1] combined with BFGS Hessian approximation [10]. In addition, motivated by homotopy based optimization methods [], we designed the sequence of the weight parameters in $f_{\text{geo}}(q)$ and $f_{\text{phys}}(q)$ such as $w_{\text{eq}}$, $w_{\text{cp}}$ and iteratively solve the bi-level optimization problem (2) to induce a better local minimum.

### *C. Lower-Level Problem Optimization with ADMM*

For a fixed $q$, the lower-level problem is a convex optimization problem with respect to $\lambda$ which can be efficiently solved. Moreover, exploiting the inherent structure of the lower-level problem, we can reformulate it to be separable with respect to the objects $\mathcal{P}_{k+1}$ in the scene and apply the alternating direction method of multipliers (ADMM) [3] to solve it in a scalable manner. For each contact force $\lambda_j$, we introduce two local copies $\lambda_{j,1}$ and $\lambda_{j,2}$ associated with the two objects in contact. Specifically, for $j \in I_{l,m}$ with $l < m$, $\lambda_{j,1}$ and $\lambda_{j,2}$ are associated with $O_l$ and $O_m$, respectively. We then impose the consensus constraints $\lambda_{j,1} = \lambda_{j,2}$. Collecting all local copies into $\tilde{\lambda}$ makes the force and torque equilibrium term separable by body: each body's residual depends only on its own local contact-force variables. The consensus constraints maintain agreement between copies of each shared contact force. The lower-level problem can therefore be reformulated as

$$\begin{aligned} \min_{\tilde{\lambda}, z} \quad & \tilde{f}_{\text{phys}}(q, \tilde{\lambda}) + \sum_{\substack{j=1,\ldots,n_c \\ s=1,2}} \delta_{K_j}(z_{j,s}) \\ \text{s.t.} \quad & \lambda_{j,1} = \lambda_{j,2} = z_{j,1} = z_{j,2}, \qquad j = 1, \ldots, n_c. \end{aligned}$$

where $\tilde{f}_{\text{phys}}(q, \tilde{\lambda})$ is the extended physical force function and $\delta_{K_j}$ is the indicator function of the second-order cone $K_j := \{\lambda \in \mathbb{R}^3 \mid g_{\text{cone},j}(q, \lambda) \leq 0\}$. Here, $z_{j,s}$ is an auxiliary variable associated with the $j$-th contact point and $s \in \{1, 2\}$. At each ADMM iteration, the objectwise-separable subproblem decomposes into $|\mathcal{P}_{k+1}|$ independent subproblems, one for each object $O_l$. The remaining updates consist of $2n_c$ projections onto the second-order cones $K_j$ which can be computed in closed form, and $2n_c$ auxiliary-variable updates, which reduce to averaging.

The pose optimization module is implemented using CUDA to leverage GPU parallelization for efficient computation of the gradients and the ADMM updates.

Visualization examples of the score maps of the initial pose sampling module $\pi_{\text{init}}$.

Fig. 4: Score maps of the initial pose sampling module $\pi_{\text{init}}$.

## V. Sequence Planning

In this section, we introduce the remaining modules $\pi_{\text{init}}$ and $\pi_{\text{sim}}$ that are used to generate candidate poses and simulate the stability of the scene, respectively. Then, we present the Monte Carlo Tree Search (MCTS) algorithm that uses these modules to explore the space of possible stacking sequences and find an optimal sequence of object placements.

### A. Initial Pose Sampling

The sampler proposes an object $i$, orientation $R$, and planar location $\mathbf{x}$ on a feasible grid $\mathcal{G}_{i,R}(s_k)$ according to

$$\pi_{\text{init}}(i, R, \mathbf{x} \mid s_k) = \pi_{xy}(\mathbf{x} \mid i, R, s_k)\pi_{\text{unif}}(i, R \mid s_k),$$
$$\pi_{xy}(\mathbf{x} \mid i, R, s_k) \propto \exp(\sigma(\mathbf{x} \mid i, R, s_k)/\tau)\,.$$

where $\pi_{\text{unif}}$ is uniform over the remaining objects $i \in \mathcal{I}_k$ and finite rotation set $\mathcal{R}$, and $\tau > 0$ controls the preference for high scores. Scene geometry determines the initial height $z_{i,R}$, yielding $\hat{q}_{k+1} = (R, (\mathbf{x}, z_{i,R}(\mathbf{x}; s_k)))$ for refinement by $\pi_{\text{opt}}$. Omitting the state dependence, the proposal score is

$$\sigma(\mathbf{x} \mid i_{k+1}, R) = w_{\text{s}}\sigma_{\text{s}}(\mathbf{x} \mid i_{k+1}, R) + w_{\text{f}}\sigma_{\text{f}}(\mathbf{x} \mid i_{k+1}, R) + w_{\text{b}}\sigma_{\text{b}}(\mathbf{x} \mid i_{k+1}, R) - w_{\text{h}}\sigma_{\text{h}}(\mathbf{x} \mid i_{k+1}, R),$$

with positive weights. The support score $\sigma_{\text{s}}$ favors larger support areas, $\sigma_{\text{f}}$ rewards filling empty floor area, $\sigma_{\text{b}}$ favors the target boundary, and $\sigma_{\text{h}}$ penalizes height. The floor and boundary terms are inspired by the dense first-course packing and boundary-first behavior observed by Menezes et al. [20]; the height penalty follows the low-placement preference of their geometric baseline. We use these preferences to sample initial poses for continuous refinement. Figure 4 illustrates these score maps; their computation is deferred to the appendix.

### B. Simulation Module

After $\pi_{\text{opt}}$ refines the candidate pose $\hat{q}_{k+1}$, the simulation module $\pi_{\text{sim}}$ evaluates the deterministic transition $s_{k+1} = F(s_k, a_k)$. It first simulates the new object with the existing scene fixed to refine the placement, and then simulates the entire scene to verify global stability.

Both stages use early termination. The simulation stops when all placed objects have sufficiently small kinetic energy or when accumulated motion energy indicates instability. For object $O_i$, we define

$$E_{\text{kin},i}(t) = \frac{1}{2m_i}\mathcal{V}_i^\top M_i \mathcal{V}_i, \quad E_{\text{acc},i}(t) = \sum_{s=0}^{t} \sqrt{E_{\text{kin},i}(s)}\Delta t$$

where $\mathcal{V}_i$ is the spatial velocity, $M_i$ and $m_i$ are the mass matrix and mass, respectively, and $\Delta t$ is the simulation time step. The early-termination condition is

$$\max_{i \in \mathcal{P}_{k+1}} E_{\text{kin},i}(t) < \epsilon_{\text{kin}} \quad \text{or} \quad \max_{i \in \mathcal{P}_{k+1}} E_{\text{acc},i}(t) > \epsilon_{\text{acc}} \tag{5}$$

with the thresholds $\epsilon_{\text{kin}}, \epsilon_{\text{acc}} > 0$. We implement $\pi_{\text{sim}}$ in CUDA using the GPU-accelerated subADMM algorithm [11].

### C. Reward Design

The reward balances target coverage, object containment, surface uniformity, and physical stability:

$$r(s_k, a_k) = w_T \frac{\mathbb{V}\left(\overline{\mathcal{X}}_{k+1}\right) - \mathbb{V}\left(\overline{\mathcal{X}}_k\right)}{\mathbb{V}(T)} + w_O \frac{\mathbb{V}\left(O_{i_{k+1}} \cap T\right)}{\mathbb{V}\left(O_{i_{k+1}}\right)} - w_H \left(\max_{\mathbf{x}} H_{\mathcal{X}_{k+1}}(\mathbf{x}) - \mathbb{E}_{\mathbf{x}}[H_{\mathcal{X}_{k+1}}(\mathbf{x})]\right) - w_D \mathbb{E}_{\mathcal{V} \sim \mathcal{N}(0,\Sigma)}\left[\Delta q_{k+1}(\mathcal{V})\right] - w_S f_{\text{eq}}$$

**Algorithm 1** Batched receding-horizon MCTS for sequence planning

**Require:** State $s_k$, previous search tree $\mathcal{T}$, batch size $B$
**Ensure:** Next state $s_{k+1}$ and updated tree $\mathcal{T}$

```
Initialize or update T at s_k and evaluate root candidates
while the search budget remains do
    B ← SELECTBATCH(T, B)
    E ← EVALUATEBATCH(B, π_init, π_opt, π_sim)
    T ← BACKUP(T, B, E)
end while
a* ← arg max_{a ∈ A_val(s_k)} Q(n_a)
Execute a* and obtain s_{k+1}
Promote its child to the root and retain compatible descendants
return s_{k+1}, T
```

where all weights are positive, $\overline{\mathcal{X}}_k := T \cap \bigcup_{i \in \mathcal{P}_k} O_i(q_i)$ is the occupied target region, and $H_{\mathcal{X}_{k+1}}(\mathbf{x})$ is the upper surface height at $\mathbf{x} = (x, y)$. The first two terms reward newly filled target volume and the fraction of the candidate contained in the target, respectively. The remaining terms penalize surface height variation, the candidate's expected pose deviation $\Delta q_{k+1}(\mathcal{V})$ under Gaussian perturbations $\mathcal{V} \sim \mathcal{N}(0, \Sigma)$, and the equilibrium residual $f_{\text{eq}}$ from $\pi_{\text{opt}}$.

### D. Monte Carlo Tree Search

We combine $\pi_{\text{init}}$, $\pi_{\text{opt}}$, and $\pi_{\text{sim}}$ in a batched, receding-horizon MCTS planner over the MDP defined in Sec. III. To amortize expensive pose optimization and simulation, we evaluate candidate placements in parallel, following the batched planning approach of [8]. Algorithm 1 summarizes the finite-horizon search and execution procedure, starting from a tree rooted at the current state $s_k$.

**Batched Selection.** We select a batch of expansion paths using PUCT [21] with geometry- and equilibrium-informed action priors. For a child $n$ reached from $n_p$ by action $a$, selection maximizes

$$\text{PUCT}(n) = \frac{N(n)Q(n)}{N(n) + \hat{N}(n)} + cP(n_p, a)\frac{\sqrt{N(n_p) + \hat{N}(n_p)}}{1 + N(n) + \hat{N}(n)}, \tag{6}$$

where $Q$ is the mean return, $N$ and $\hat{N}$ are completed and temporary virtual visit counts, $P$ is the action prior, and $c > 0$ controls exploration. Progressive widening allows more children as completed visits increase, while virtual visits diversify selection within each batch. All paths are selected before physical evaluation begins.

**Parallel Expansion and Evaluation.** Each selected expansion applies $\pi_{\text{init}}$, $\pi_{\text{opt}}$, and $\pi_{\text{sim}}$ to generate, refine, and physically evaluate a candidate placement. These evaluations run in parallel and provide settled successor states, validity outcomes, and rewards for tree expansion and backup. Root candidates are evaluated through the same pipeline.

**Leaf Value Estimation and Backup.** To avoid further optimization and simulation costs, we replace additional rollouts

with a heuristic estimate of the remaining return. For a nonterminal leaf $s_l$ at depth $d_l$ from the current root, we use

$$\hat{V}(s_l) = \bar{r} \sum_{j=0}^{h_l - 1} \gamma^j, \qquad h_l = \min\{d_M - d_l, |\mathcal{I}(s_l)|\},$$

where $d_M$ is the maximum search depth, $\mathcal{I}(s_l)$ is the remaining inventory, and $\bar{r}$ is the mean reward estimated from evaluated placements. Terminal and failed leaves receive no bootstrap value. Discounted placement rewards and the leaf estimate are backed up along each selected path to update mean returns $Q$ and visit counts $N$. Temporary virtual visits are then removed.

**Action Selection and Tree Reuse.** After the search budget is exhausted, we execute the physically validated root action with the highest mean return. In Algorithm 1, $\mathcal{A}_{\text{val}}(s_k)$ denotes these validated actions and $n_a$ the child reached by $a$. The selected child becomes the new root, retaining descendants and search statistics compatible with the resulting state. Planning then repeats with a finite lookahead horizon.

## Acknowledgments

## References


[1] P. A. Absil, C. G. Baker, and K. A. Gallivan. Trust-region methods on riemannian manifolds. *Foundations of Computational Mathematics*, 7:303–330, 2007.

[2] S. An, S. Lee, J. Lee, S. Park, and D. Lee. Collision detection between smooth convex bodies via riemannian optimization framework. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12464–12471. IEEE, 2024.

[3] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011. doi: 10.1561/2200000016.

[4] M. Carter. *Foundations of mathematical economics*. MIT press, 2001.

[5] J. Chen, Y. Ke, and H. Wang. BODex: Scalable and efficient robotic dexterous grasp synthesis using bilevel optimization. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1–8. IEEE, 2025. doi: 10.1109/ICRA55743.2025.11127930.

[6] K. A. Dowsland. Efficient automated pallet loading. *European Journal of Operational Research*, 44(2):232–238, 1990. doi: 10.1016/0377-2217(90)90358-I.

[7] F. Furrer, M. Wermelinger, H. Yoshida, F. Gramazio, M. Kohler, R. Siegwart, and M. Hutter. Autonomous robotic stone stacking with online next best object target pose planning. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2350–2356, 2017. doi: 10.1109/ICRA.2017.7989272.

[8] B. Huang, A. Boularias, and J. Yu. Parallel monte carlo tree search with batched rigid-body simulations for speeding up long-horizon episodic robot planning. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022.

[9] S. Huang, Z. Wang, J. Zhou, and J. Lu. Planning irregular object packing via hierarchical reinforcement learning. *IEEE Robotics and Automation Letters*, 8(1): 81–88, 2023. doi: 10.1109/LRA.2022.3222996.

[10] W. Huang, P.-A. Absil, and K. A. Gallivan. A riemannian BFGS method without differentiated retraction for nonconvex optimization problems. *SIAM Journal on Optimization*, 28(1):470–495, 2018. doi: 10.1137/17M1127582.

[11] H. Ji, H. Kim, J. Lee, S. Lee, S. An, J. Heo, Y. Lee, Y. Lee, and D. Lee. Gpu-accelerated subsystem-based admm for large-scale interactive simulation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 13868–13874. IEEE, 2025.

[12] R. L. Johns, M. Wermelinger, R. Mascaro, D. Jud, F. Gramazio, M. Kohler, M. Chli, and M. Hutter. Autonomous dry stone. *Construction Robotics*, 4(3–4):127–140, 2020. doi: 10.1007/s41693-020-00037-6.

[13] R. L. Johns, M. Wermelinger, R. Mascaro, D. Jud, I. Hurkxkens, L. Vasey, M. Chli, F. Gramazio, M. Kohler, and M. Hutter. A framework for robotic excavation and dry stone construction using on-site materials. *Science Robotics*, 8(84):eabp9758, 2023. doi: 10.1126/scirobotics.abp9758.

[14] J. E. King, V. Ranganeni, and S. S. Srinivasa. Unobservable monte carlo planning for nonprehensile rearrangement tasks. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4681–4688, 2017. doi: 10.1109/ICRA.2017.7989544.

[15] J. Lee, M. Lee, and D. Lee. Uncertain pose estimation during contact tasks using differentiable contact features. *RSS 2023-Robotics: Science and Systems*, 2023.

[16] J. Lee, S. Park, M. Lee, and D. Lee. Efficient gradient-based inference for manipulation planning in contact factor graphs. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 12464–12471. IEEE, 2025.

[17] Y. Liu and N. Napp. Stability-based sequence planning for robotic dry-stacking of natural stones. *IEEE Robotics and Automation Letters*, 8(9):5894–5901, 2023. doi: 10.1109/LRA.2023.3301275.

[18] Y. Liu and N. Napp. Robotic dry-stacking of clocháin with irregular stones. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 11517–11523, 2025. doi: 10.1109/ICRA55743.2025.11128654.

[19] Y. Liu, J. Choi, and N. Napp. Planning for robotic dry stacking with irregular stones. In G. Ishigami and K. Yoshida, editors, *Field and Service Robotics*, volume 16 of *Springer Proceedings in Advanced Robotics*, pages 321–335. Springer, 2021. doi: 10.1007/978-981-15-9460-1_23.

[20] A. Menezes, P. Vicente, A. Bernardino, and R. Ventura. From rocks to walls: A model-free reinforcement learning approach to dry stacking with irregular rocks. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 2057–2065, 2021. doi: 10.1109/CVPRW53098.2021.00234.

[21] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis. Mastering the game of go without human knowledge. *Nature*, 550:354–359, 2017.

[22] F. Wang and K. Hauser. Dense robotic packing of irregular and novel 3D objects. *IEEE Transactions on Robotics*, 38(2):1160–1173, 2022. doi: 10.1109/TRO.2021.3097261.